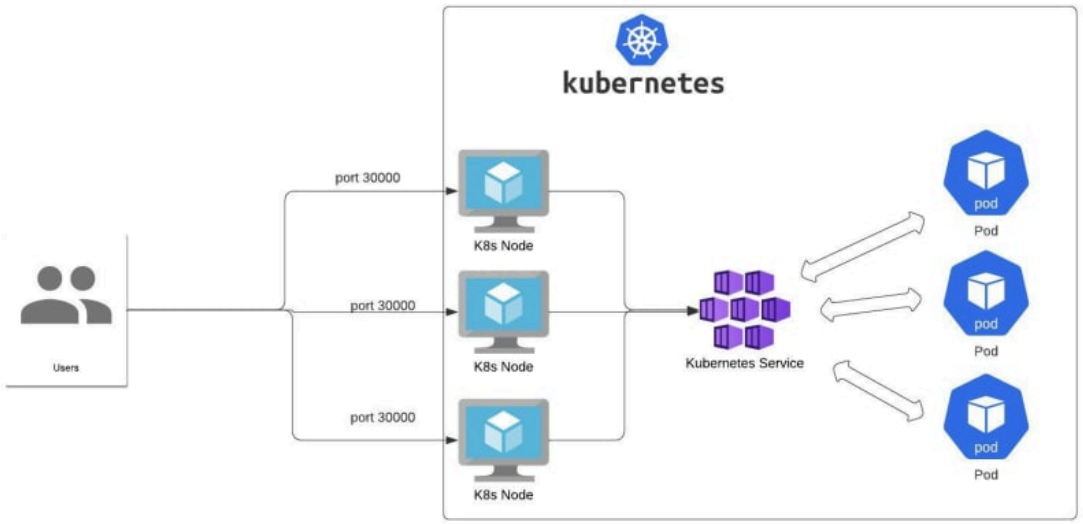
Despliegue de Kubernetes (k3s) con Balanceo de Carga y Conexión Híbrida AWS

Alumno: José Rosa López

Asignatura: Arquitectura en la Nube

Curso: 2º ASIR



Índice

Conceptos Clave antes de empezar

- **Kubernetes (k3s):** Sistema para gestionar aplicaciones en contenedores. Usaremos k3s, una versión ligera de 50MB ideal para WSL2.
- **Pod:** La unidad mínima (un contenedor ejecutándose).
- **Deployment:** Se asegura de que siempre haya un número específico de Pods vivos (si uno muere, crea otro).
- **Service (Load Balancer):** Provee una IP estable y distribuye el tráfico entre los Pods.

Requisitos Previos

Asegúrate de tener:

1. Windows 11 con WSL2 (Ubuntu 22.04 o superior).
2. Una cuenta de AWS activa (Free Tier).
3. Tu clave `.pem` de AWS lista para usar.

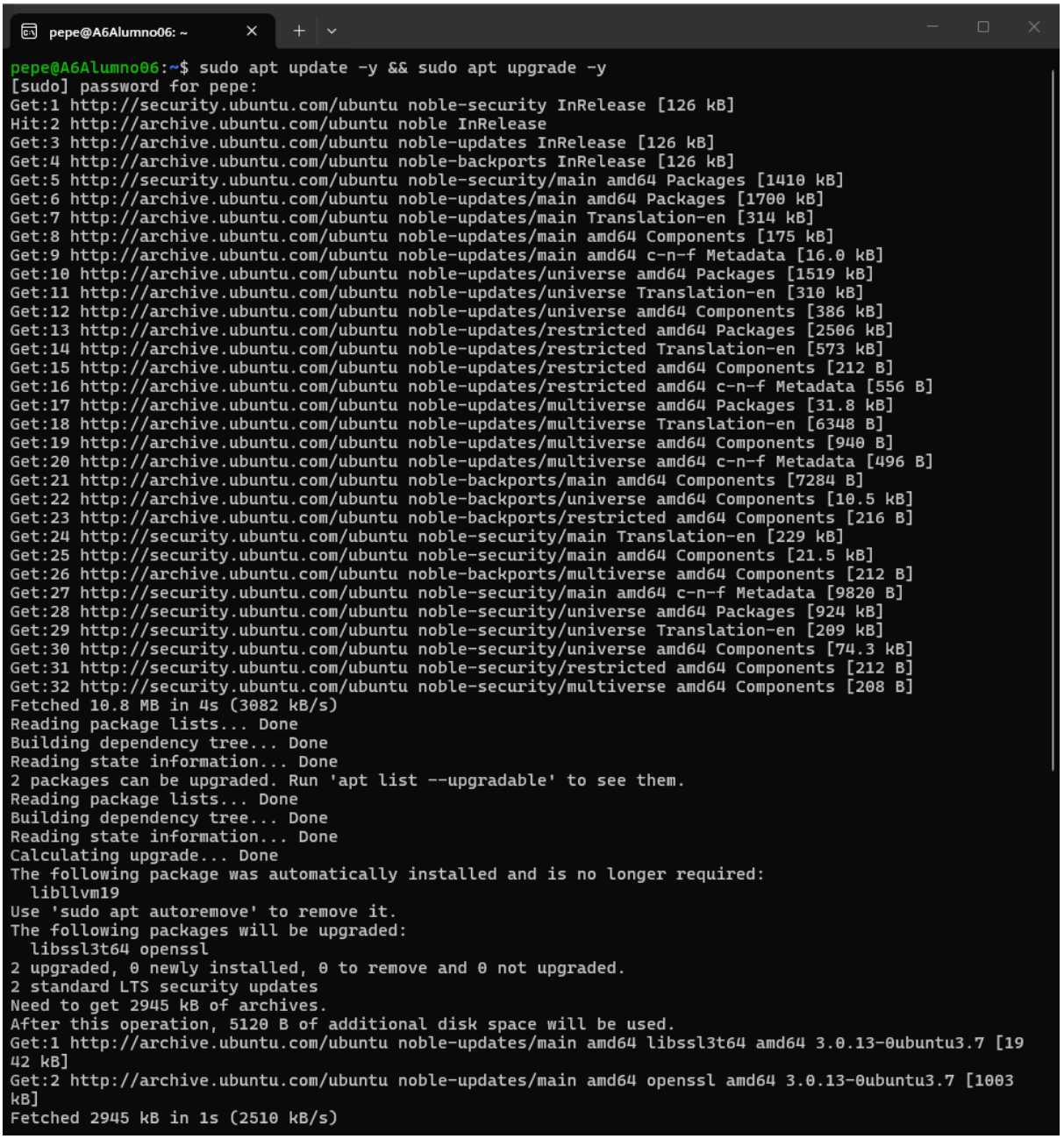
PARTE 0: Preparación del Entorno (WSL2)

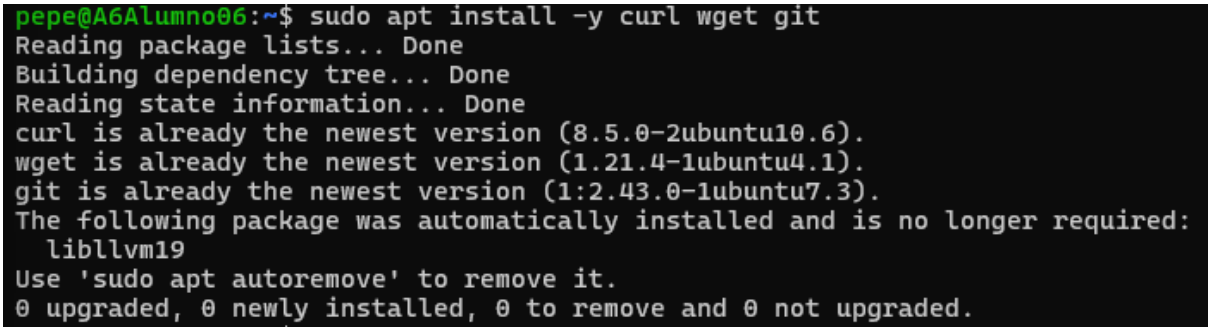
En esta fase instalaremos Kubernetes sin usar Docker, para evitar complicaciones y consumo excesivo de recursos.

0.1 Instalar k3s

Abre tu terminal de Ubuntu en WSL2 y ejecuta:

Actualizar el sistema e instalar herramientas básicas:
Bash
sudo apt update -y && sudo apt upgrade -y
sudo apt install -y curl wget git





Instalar k3s (sin systemd): Este comando instala k3s configurado para que cualquier usuario pueda leer la configuración (`mode 644`).

Bash

```
curl -sfL https://get.k3s.io | K3S_KUBECONFIG_MODE="644" sh -
```

```
pepe@A6Alumno06:~$ curl -sfL https://get.k3s.io | K3S_KUBECONFIG_MODE="644" sh -
[INFO]  Finding release for channel stable
[INFO]  Using v1.34.3+k3s1 as release
[INFO]  Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.34.3+k3s1/sha256sum-amd64.
txt
[INFO]  Skipping binary downloaded, installed k3s matches hash
[INFO]  Skipping installation of SELinux RPM
[INFO]  Skipping /usr/local/bin/kubectl symlink to k3s, already exists
[INFO]  Skipping /usr/local/bin/crictl symlink to k3s, already exists
[INFO]  Skipping /usr/local/bin/ctr symlink to k3s, already exists
[INFO]  Creating killall script /usr/local/bin/k3s-killall.sh
[INFO]  Creating uninstall script /usr/local/bin/k3s-uninstall.sh
[INFO]  env: Creating environment file /etc/systemd/system/k3s.service.env
[INFO]  systemd: Creating service file /etc/systemd/system/k3s.service
[INFO]  systemd: Enabling k3s unit
Created symlink /etc/systemd/system/multi-user.target.wants/k3s.service → /etc/systemd/system/k3s.serv
ice.
[INFO]  No change detected so skipping service start
```

Iniciar el servidor k3s en segundo plano:

Bash

sudo k3s server & sleep 10

```
pepe@A6Alumno06: ~
pepe@A6Alumno06:~$ sudo k3s server & sleep 10
[1] 2683
INFO[0000] Starting k3s v1.34.3+k3s1 (48ffa7b6)
INFO[0000] Configuring sqlite3 database connection pooling: maxIdleConns=2, maxOpenConns=0, connMaxLif
etime=0s
INFO[0000] Configuring database table schema and indexes, this may take a moment...
INFO[0000] Database tables and indexes are up to date
INFO[0000] Kine available at unix://kine.sock
INFO[0000] Reconciling bootstrap data between datastore and disk
INFO[0000] Password verified locally for node a6alumno06
INFO[0000] certificate CN=a6alumno06 signed by CN=k3s-server-ca@1769431047: notBefore=2026-01-26 12:37
:27 +0000 UTC notAfter=2027-01-28 08:06:38 +0000 UTC
INFO[0000] certificate CN=system:node:a6alumno06,O=system:nodes signed by CN=k3s-client-ca@1769431047:
 notBefore=2026-01-26 12:37:27 +0000 UTC notAfter=2027-01-28 08:06:38 +0000 UTC
INFO[0000] certificate CN=system:kube-proxy signed by CN=k3s-client-ca@1769431047: notBefore=2026-01-2
6 12:37:27 +0000 UTC notAfter=2027-01-28 08:06:38 +0000 UTC
INFO[0000] Module overlay was already loaded
INFO[0000] Module nf_conntrack was already loaded
INFO[0000] Module br_netfilter was already loaded
INFO[0000] Module iptable_nat was already loaded
INFO[0000] Module iptable_filter was already loaded
WARN[0000] Failed to load kernel module nft-expr-counter with modprobe
INFO[0000] Connecting to proxy                           url="wss://127.0.0.1:6443/v1-k3s/connect"
INFO[0000] Logging containerd to /var/lib/rancher/k3s/agent/containerd/containerd.log
INFO[0000] Running containerd -c /var/lib/rancher/k3s/agent/etc/containerd/config.toml
INFO[0000] Creating k3s-cert-monitor event broadcaster
INFO[0000] Running kube-apiserver --advertise-port=6443 --allow-privileged=true --anonymous-auth=false
 --api-audiences=https://kubernetes.default.svc.cluster.local,k3s --authorization-mode=Node,RBAC --bin
d-address=127.0.0.1 --cert-dir=/var/lib/rancher/k3s/server/tls/temporary-certs --client-ca-file=/var/l
ib/rancher/k3s/server/tls/client-ca.crt --egress-selector-config-file=/var/lib/rancher/k3s/server/etc/
egress-selector-config.yaml --enable-admission-plugins=NodeRestriction --enable-aggregator-routing=tru
e --enable-bootstrap-token-auth=true --etcd-servers=unix://kine.sock --kubelet-certificate-authority=/
var/lib/rancher/k3s/server/tls/server-ca.crt --kubelet-client-certificate=/var/lib/rancher/k3s/server/
tls/client-kube-apiserver.crt --kubelet-client-key=/var/lib/rancher/k3s/server/tls/client-kube-apiserv
er.key --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname --profiling=false --proxy-clie
nt-cert-file=/var/lib/rancher/k3s/server/tls/client-auth-proxy.crt --proxy-client-key-file=/var/lib/ra
ncher/k3s/server/tls/client-auth-proxy.key --requestheader-allowed-names=system:auth-proxy --requesthe
ader-client-ca-file=/var/lib/rancher/k3s/server/tls/request-header-ca.crt --requestheader-extra-header
s-prefix=X-Remote-Extra- --requestheader-group-headers=X-Remote-Group --requestheader-username-headers
=X-Remote-User --secure-port=6444 --service-account-issuer=https://kubernetes.default.svc.cluster.loca
l --service-account-key-file=/var/lib/rancher/k3s/server/tls/service.key --service-account-signing-key
-file=/var/lib/rancher/k3s/server/tls/service.current.key --service-cluster-ip-range=10.43.0.0/16 --se
rvice-node-port-range=30000-32767 --storage-backend=etcd3 --tls-cert-file=/var/lib/rancher/k3s/server/
tls/serving-kube-apiserver.crt --tls-cipher-suites=TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384,TLS_ECDHE_R
SA_WITH_AES_256_GCM_SHA384,TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA2
56,TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305,TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305 --tls-private-key-file=
/var/lib/rancher/k3s/server/tls/serving-kube-apiserver.key
INFO[0000] Connection to etcd is ready
INFO[0000] ETCD server is now running
INFO[0000] Running kube-scheduler --authentication-kubeconfig=/var/lib/rancher/k3s/server/cred/schedul
er.kubeconfig --authorization-kubeconfig=/var/lib/rancher/k3s/server/cred/scheduler.kubeconfig --bind-
address=127.0.0.1 --kubeconfig=/var/lib/rancher/k3s/server/cred/scheduler.kubeconfig --leader-elect=fa
lse --profiling=false --secure-port=10259 --tls-cert-file=/var/lib/rancher/k3s/server/tls/kube-schedul
er/kube-scheduler.crt --tls-private-key-file=/var/lib/rancher/k3s/server/tls/kube-scheduler/kube-sched
uler.key
INFO[0000] Running kube-controller-manager --allocate-node-cidrs=true --authentication-kubeconfig=/var
/lib/rancher/k3s/server/cred/controller.kubeconfig --authorization-kubeconfig=/var/lib/rancher/k3s/ser
```

Verificar que funciona:
Bash
sudo k3s kubectl get nodes

```
pepe@A6Alumno06:~$ sudo k3s kubectl get nodes
NAME          STATUS   ROLES           AGE   VERSION
a6alumno06    Ready    control-plane   43h   v1.34.3+k3s1
```

1. *Deberías ver tu nodo en estado "Ready"*.

0.2 Configurar `kubectl` local

Para no tener que usar `sudo k3s kubectl` todo el tiempo, instalaremos el cliente oficial.

Descargar e instalar el binario:
Bash
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo chmod +x kubectl
sudo mv kubectl /usr/local/bin/

```
pepe@A6Alumno06:~$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.t
xt)/bin/linux/amd64/kubectl"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   138  100   138    0     0    726      0 --:--:-- --:--:-- --:--:--   730
100 55.8M  100 55.8M    0     0  65.8M      0 --:--:-- --:--:-- --:--:--  100M
pepe@A6Alumno06:~$ sudo chmod +x kubectl
pepe@A6Alumno06:~$ sudo mv kubectl /usr/local/bin/
```

Configurar el acceso: Copiamos la configuración de k3s a tu carpeta de usuario.
Bash
mkdir -p $HOME/.kube
sudo cp /etc/rancher/k3s/k3s.yaml $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
sudo chmod 600 $HOME/.kube/config

```
pepe@A6Alumno06:~$ mkdir -p $HOME/.kube
pepe@A6Alumno06:~$ sudo cp /etc/rancher/k3s/k3s.yaml $HOME/.kube/config
pepe@A6Alumno06:~$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
pepe@A6Alumno06:~$ sudo chmod 600 $HOME/.kube/config
```

Prueba final: Ejecuta `kubectl get nodes`. Si responde sin pedir contraseña, está listo.

```
pepe@A6Alumno06:~$ kubectl get nodes
NAME          STATUS   ROLES           AGE   VERSION
a6alumno06    Ready    control-plane   43h   v1.34.3+k3s1
```

0.3 Crear directorio de trabajo
Bash
mkdir -p ~/kubernetes-aws-practice

cd ~/kubernetes-aws-practice

```
pepe@A6Alumno06:~$ mkdir -p ~/kubernetes-aws-practice
pepe@A6Alumno06:~$ cd ~/kubernetes-aws-practice
pepe@A6Alumno06:~/kubernetes-aws-practice$
```

PARTE 1: Crear la Aplicación

Vamos a crear una aplicación web simple en Python que nos diga qué "Pod" nos está atendiendo.

Crear carpeta de la app:
Bash
mkdir -p ~/kubernetes-aws-practice/app
cd ~/kubernetes-aws-practice/app

```
pepe@A6Alumno06:~/kubernetes-aws-practice$ mkdir -p ~/kubernetes-aws-practice/app
pepe@A6Alumno06:~/kubernetes-aws-practice$ cd ~/kubernetes-aws-practice/app
pepe@A6Alumno06:~/kubernetes-aws-practice/app$
```

1.

Crear el archivo `app.py`: Copia y pega este bloque entero en tu terminal. Este código crea un servidor Flask que muestra el nombre del pod y el namespace .
Bash

```
cat > app.py << 'EOF'
#!/usr/bin/env python3
from flask import Flask, jsonify, send_from_directory
import os
import socket
from datetime import datetime
import sys

app = Flask(__name__)

# Variables de entorno inyectadas por Kubernetes
POD_NAME = os.getenv('POD_NAME', 'Unknown Pod')
POD_NAMESPACE = os.getenv('POD_NAMESPACE', 'default')

@app.route('/')
def index():
    return send_from_directory('.', 'index.html')

@app.route('/pod-info')
def pod_info():
    return jsonify({
        'pod_name': POD_NAME,
        'namespace': POD_NAMESPACE,
        'hostname': socket.gethostname(),
```

```
        'timestamp': datetime.now().isoformat()
    })

@app.route('/health')
def health():
    return jsonify({'status': 'healthy', 'pod': POD_NAME}), 200

if __name__ == '__main__':
    print(f"[{POD_NAME}] Iniciando servidor Flask...", file=sys.stderr)
    app.run(host='0.0.0.0', port=5000, debug=False)
EOF
```

```
pepe@A6Alumno06:~/kubernetes-aws-practice/app$ cat > app.py << 'EOF'
#!/usr/bin/env python3
from flask import Flask, jsonify, send_from_directory
import os
import socket
from datetime import datetime
import sys

app = Flask(__name__)

# Variables de entorno inyectadas por Kubernetes
POD_NAME = os.getenv('POD_NAME', 'Unknown Pod')
POD_NAMESPACE = os.getenv('POD_NAMESPACE', 'default')

@app.route('/')
def index():
    return send_from_directory('.', 'index.html')

@app.route('/pod-info')
def pod_info():
    return jsonify({
        'pod_name': POD_NAME,
        'namespace': POD_NAMESPACE,
        'hostname': socket.gethostname(),
        'timestamp': datetime.now().isoformat()
    })

@app.route('/health')
def health():
    return jsonify({'status': 'healthy', 'pod': POD_NAME}), 200

if __name__ == '__main__':
    print(f"[{POD_NAME}] Iniciando servidor Flask...", file=sys.stderr)
    app.run(host='0.0.0.0', port=5000, debug=False)
EOF
```

2. Dale permisos de ejecución: `sudo chmod +x app.py`.

```
pepe@A6Alumno06:~/kubernetes-aws-practice/app$ sudo chmod +x app.py
```

Crear el archivo `index.html`: Este HTML mostrará visualmente el balanceo de carga .
(Ejecuta el bloque siguiente en tu terminal):

Bash

```
cat > index.html << 'EOF'
<!DOCTYPE html>
<html>
<head>
    <title>Kubernetes Load Balancer</title>
    <style>
        body { font-family: Arial, sans-serif; display: flex; justify-content: center; align-items: center; min-height: 100vh; margin: 0; background: linear-gradient(135deg, #667eea 0%, #764ba2 100%); }
        .container { background: white; padding: 50px; border-radius: 10px; box-shadow: 0 10px 25px rgba(0,0,0,0.2); text-align: center; max-width: 500px; }
        h1 { color: #667eea; margin: 0 0 30px 0; }
        .info { background: #f0f0f0; padding: 20px; border-radius: 5px; margin: 20px 0; }
        .pod-name { font-size: 28px; color: #764ba2; font-weight: bold; font-family: monospace; }
        .timestamp { color: #666; font-size: 13px; margin-top: 10px; }
        .instruction { background: #e0f2fe; padding: 15px; border-radius: 5px; color: #0369a1; font-size: 14px; margin-top: 20px; }
        .refresh-button { margin-top: 20px; padding: 10px 20px; background: #667eea; color: white; border: none; border-radius: 5px; cursor: pointer; font-size: 14px; }
        .refresh-button:hover { background: #764ba2; }
    </style>
</head>
<body>
    <div class="container">
        <h1>Kubernetes Load Balancer</h1>
        <div class="info">
            <p style="margin: 0 0 10px 0; color: #666;">Pod atendiendo solicitud:</p>
            <p class="pod-name" id="pod-name">Cargando...</p>
            <p class="timestamp" id="timestamp"></p>
        </div>
        <div class="instruction">Actualiza constantemente esta página para ver el <b>balanceo de carga en acción</b></div>
        <button class="refresh-button" onclick="location.reload()">Actualizar Ahora</button>
    </div>
    <script>
        fetch('/pod-info')
            .then(r => r.json())
            .then(data => {
                document.getElementById('pod-name').textContent = data.pod_name;
                const date = new Date(data.timestamp);
                document.getElementById('timestamp').textContent = date.toLocaleString('es-ES');
            })
            .catch(e => {
                document.getElementById('pod-name').textContent = 'Error';
                console.error('Error:', e);
            });
```

```
    </script>
</body>
</html>
EOF
```

```
pepe@A6Alumno06:~/kubernetes-aws-practice/app$ cat > index.html << 'EOF'
<!DOCTYPE html>
<html>
<head>
    <title>Kubernetes Load Balancer</title>
    <style>
        body { font-family: Arial, sans-serif; display: flex; justify-content: center; align-items: ce
nter; min-height: 100vh; margin: 0; background: linear-gradient(135deg, #667eea 0%, #764ba2 100%); }
        .container { background: white; padding: 50px; border-radius: 10px; box-shadow: 0 10px 25px rg
ba(0,0,0,0.2); text-align: center; max-width: 500px; }
        h1 { color: #667eea; margin: 0 0 30px 0; }
        .info { background: #f0f0f0; padding: 20px; border-radius: 5px; margin: 20px 0; }
        .pod-name { font-size: 28px; color: #764ba2; font-weight: bold; font-family: monospace; }
        .timestamp { color: #666; font-size: 13px; margin-top: 10px; }
        .instruction { background: #e0f2fe; padding: 15px; border-radius: 5px; color: #0369a1; font-si
ze: 14px; margin-top: 20px; }
        .refresh-button { margin-top: 20px; padding: 10px 20px; background: #667eea; color: white; bor
der: none; border-radius: 5px; cursor: pointer; font-size: 14px; }
        .refresh-button:hover { background: #764ba2; }
    </style>
</head>
<body>
    <div class="container">
        <h1>Kubernetes Load Balancer</h1>
        <div class="info">
            <p style="margin: 0 0 10px 0; color: #666;">Pod atendiendo solicitud:</p>
            <p class="pod-name" id="pod-name">Cargando...</p>
            <p class="timestamp" id="timestamp"></p>
        </div>
        <div class="instruction">Actualiza constantemente esta página para ver el <b>balanceo de carga
 en acción</b></div>
        <button class="refresh-button" onclick="location.reload()">Actualizar Ahora</button>
    </div>
    <script>
        fetch('/pod-info')
            .then(r => r.json())
            .then(data => {
                document.getElementById('pod-name').textContent = data.pod_name;
                const date = new Date(data.timestamp);
                document.getElementById('timestamp').textContent = date.toLocaleString('es-ES');
            })
            .catch(e => {
                document.getElementById('pod-name').textContent = 'Error';
                console.error('Error:', e);
            });
    </script>
</body>
</html>
EOF
```

Crear `requirements.txt` y `Dockerfile`: Aunque k3s usará los archivos directamente mediante ConfigMaps, creamos estos ficheros como referencia estándar .

Bash

```
cat > requirements.txt << 'EOF'
Flask==3.0.0
Werkzeug==3.0.0
EOF

cat > Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
COPY app.py .
COPY index.html .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "app.py"]
```

EOF

```
pepe@A6Alumno06:~/kubernetes-aws-practice/app$ cat > requirements.txt << 'EOF'
Flask==3.0.0
Werkzeug==3.0.0
EOF

cat > Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
COPY app.py .
COPY index.html .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "app.py"]
EOF
```

PARTE 2: Configurar Kubernetes (Manifests YAML)

Aquí definiremos la infraestructura como código. Vuelve al directorio principal: `cd ~/kubernetes-aws-practice`.

```
pepe@A6Alumno06:~/kubernetes-aws-practice/app$ cd ~/kubernetes-aws-practice
pepe@A6Alumno06:~/kubernetes-aws-practice$
```

2.1 Crear Namespace

Esto aísla nuestra práctica del resto del sistema.

Bash
cat > namespace.yaml << 'EOF'
apiVersion: v1
kind: Namespace
metadata:
  name: load-balancer-demo
  labels:
    name: load-balancer-demo
EOF

kubectl apply -f namespace.yaml

```
pepe@A6Alumno06:~/kubernetes-aws-practice$ cat > namespace.yaml << 'EOF'
apiVersion: v1
kind: Namespace
metadata:
  name: load-balancer-demo
  labels:
    name: load-balancer-demo
EOF

kubectl apply -f namespace.yaml
namespace/load-balancer-demo created
```

2.2 Crear ConfigMap

Esto es un truco inteligente: en lugar de construir una imagen Docker y subirla a un registro, metemos el código (`app.py`, `index.html`) dentro de la configuración de Kubernetes. Así es más rápido editar y probar.

Copia el bloque `configmap.yaml` del guion original o usa el siguiente comando que condensa el contenido: [Nota: Debido a la longitud, asegúrate de crear el archivo `configmap.yaml` con el contenido de las páginas 11-15 del PDF].

```
pepe@A6Alumno06:~/kubernetes-aws-practice$ cat > configmap.yaml << 'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-files
  namespace: load-balancer-demo
data:
  requirements.txt: |
    Flask==3.0.0
    Werkzeug==3.0.0

  app.py: |
    #!/usr/bin/env python3
    from flask import Flask, jsonify, send_from_directory
    import os
    import socket
    from datetime import datetime
    import sys

    app = Flask(__name__)

    POD_NAME = os.getenv('POD_NAME', 'Unknown Pod')
    POD_NAMESPACE = os.getenv('POD_NAMESPACE', 'default')

    @app.route('/')
    def index():
        return send_from_directory('.', 'index.html')

    @app.route('/pod-info')
    def pod_info():
        return jsonify({
            'pod_name': POD_NAME,
            'namespace': POD_NAMESPACE,
            'hostname': socket.gethostname(),
            'timestamp': datetime.now().isoformat()
        })

    @app.route('/health')
    def health():
        return jsonify({'status': 'healthy', 'pod': POD_NAME}), 200

    if __name__ == '__main__':
        print(f"[{POD_NAME}] Iniciando servidor Flask...", file=sys.stderr)
        app.run(host='0.0.0.0', port=5000, debug=False)

  index.html: |
    <!DOCTYPE html>
    <html>
    <head>
        <title>Kubernetes Load Balancer</title>
        <style>
            body { font-family: Arial, sans-serif; display: flex; justify-content: center; align-items
: center; min-height: 100vh; margin: 0; background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
 }
            .container { background: white; padding: 50px; border-radius: 10px; box-shadow: 0 10px 25p
x rgba(0,0,0,0.2); text-align: center; max-width: 500px; }
```

Comando rápido para aplicar el ConfigMap (asegúrate de que el contenido coincida con el PDF): Ejecuta `kubectl apply -f configmap.yaml` después de crear el archivo

```
pepe@A6Alumno06:~/kubernetes-aws-practice$ kubectl apply -f configmap.yaml
configmap/app-files created
pepe@A6Alumno06:~/kubernetes-aws-practice$ kubectl get configmap -n load-balancer-demo
NAME               DATA   AGE
app-files          3      6s
kube-root-ca.crt   1      33m
```

.

2.3 Crear Deployment (3 réplicas)

Define que queremos 3 copias de nuestra app corriendo. Si una falla, Kubernetes la reinicia. Crea el archivo `deployment.yaml` .

Bash

```
cat > deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  namespace: load-balancer-demo
  labels:
    app: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: web-app
        image: python:3.11-slim
        command: ["sh", "-c"]
        args:
        - |
          cd /app
          pip install --no-cache-dir -r requirements.txt > /dev/null 2>&1
          python app.py
        ports:
        - containerPort: 5000
          protocol: TCP
        env:
        - name: POD_NAME
          valueFrom:
            fieldRef:
              fieldPath: metadata.name
        - name: POD_NAMESPACE
```

```
        valueFrom:
          fieldRef:
            fieldPath: metadata.namespace
      volumeMounts:
      - name: app-volume
        mountPath: /app
      livenessProbe:
        httpGet:
          path: /health
          port: 5000
        initialDelaySeconds: 15
        periodSeconds: 10
      readinessProbe:
        httpGet:
          path: /health
          port: 5000
        initialDelaySeconds: 5
        periodSeconds: 5
    volumes:
    - name: app-volume
      configMap:
        name: app-files
        defaultMode: 0755
EOF
```

Aplica y verifica:

Bash
```
kubectl apply -f deployment.yaml
kubectl wait --for=condition=ready pod -l app=web-app -n load-balancer-demo
--timeout=120s
kubectl get pods -n load-balancer-demo
```

```
pepe@A6Alumno06:~/kubernetes-aws-practice$ cat > deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  namespace: load-balancer-demo
  labels:
    app: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: web-app
        image: python:3.11-slim
        command: ["sh", "-c"]
        args:
        - |
          cd /app
          pip install --no-cache-dir -r requirements.txt > /dev/null 2>&1
          python app.py
        ports:
        - containerPort: 5000
          protocol: TCP
        env:
        - name: POD_NAME
          valueFrom:
            fieldRef:
              fieldPath: metadata.name
        - name: POD_NAMESPACE
          valueFrom:
            fieldRef:
              fieldPath: metadata.namespace
        volumeMounts:
        - name: app-volume
          mountPath: /app
        livenessProbe:
          httpGet:
            path: /health
            port: 5000
          initialDelaySeconds: 15
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /health
            port: 5000
          initialDelaySeconds: 5
          periodSeconds: 5
      volumes:
      - name: app-volume
```

```
pepe@A6Alumno06:~/kubernetes-aws-practice$ kubectl apply -f deployment.yaml
deployment.apps/web-app created
pepe@A6Alumno06:~/kubernetes-aws-practice$ kubectl wait --for=condition=ready pod -l app=web-app -n lo
ad-balancer-demo --timeout=120s
pod/web-app-65967466dd-5pqnt condition met
pod/web-app-65967466dd-rbk7b condition met
pod/web-app-65967466dd-wpv4g condition met
pepe@A6Alumno06:~/kubernetes-aws-practice$ kubectl get pods -n load-balancer-demo
NAME                       READY   STATUS    RESTARTS   AGE
web-app-65967466dd-5pqnt   1/1     Running   0          23s
web-app-65967466dd-rbk7b   1/1     Running   0          23s
web-app-65967466dd-wpv4g   1/1     Running   0          23s
pepe@A6Alumno06:~/kubernetes-aws-practice$
```

2.4 Crear Service (Load Balancer)

Crea el archivo `service.yaml` para distribuir el tráfico.

Bash
cat > service.yaml << 'EOF'

```
apiVersion: v1
kind: Service
metadata:
  name: web-app-service
  namespace: load-balancer-demo
  labels:
    app: web-app
spec:
  type: LoadBalancer
  selector:
    app: web-app
  ports:
  - protocol: TCP
    port: 80
    targetPort: 5000
    name: http
    sessionAffinity: None
EOF
```

kubectl apply -f service.yaml

```
pepe@A6Alumno06:~/kubernetes-aws-practice$ cat > service.yaml << 'EOF'
apiVersion: v1
kind: Service
metadata:
  name: web-app-service
  namespace: load-balancer-demo
  labels:
    app: web-app
spec:
  type: LoadBalancer
  selector:
    app: web-app
  ports:
  - protocol: TCP
    port: 80
    targetPort: 5000
    name: http
  sessionAffinity: None
EOF
pepe@A6Alumno06:~/kubernetes-aws-practice$ kubectl apply -f service.yaml
service/web-app-service created
pepe@A6Alumno06:~/kubernetes-aws-practice$
```

PARTE 3: Verificar Balanceo Local

Exponer el servicio: Como estamos en local, usamos `port-forward` para acceder al LoadBalancer interno.

Bash

kubectl port-forward -n load-balancer-demo svc/web-app-service 8080:80

```
pepe@A6Alumno06:~/kubernetes-aws-practice$ kubectl port-forward -n load-balancer-demo svc/web-app-serv
ice 8080:80
Forwarding from 127.0.0.1:8080 -> 5000
Forwarding from [::1]:8080 -> 5000
```

1. Deja esta terminal abierta.

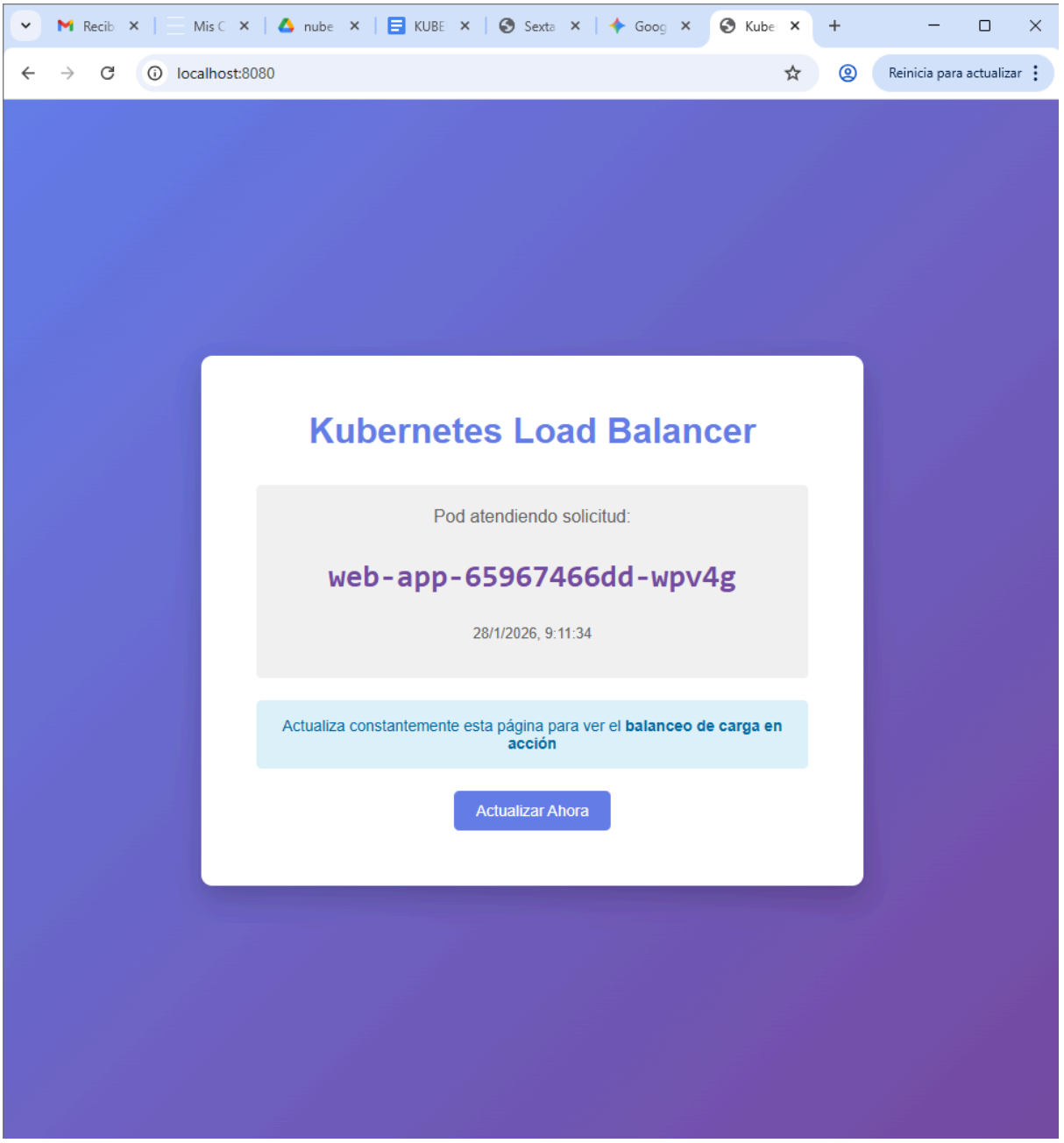
Probar (en otra terminal): Abre otra terminal de WSL y ejecuta este script para ver cómo cambia el nombre del pod (balanceo):

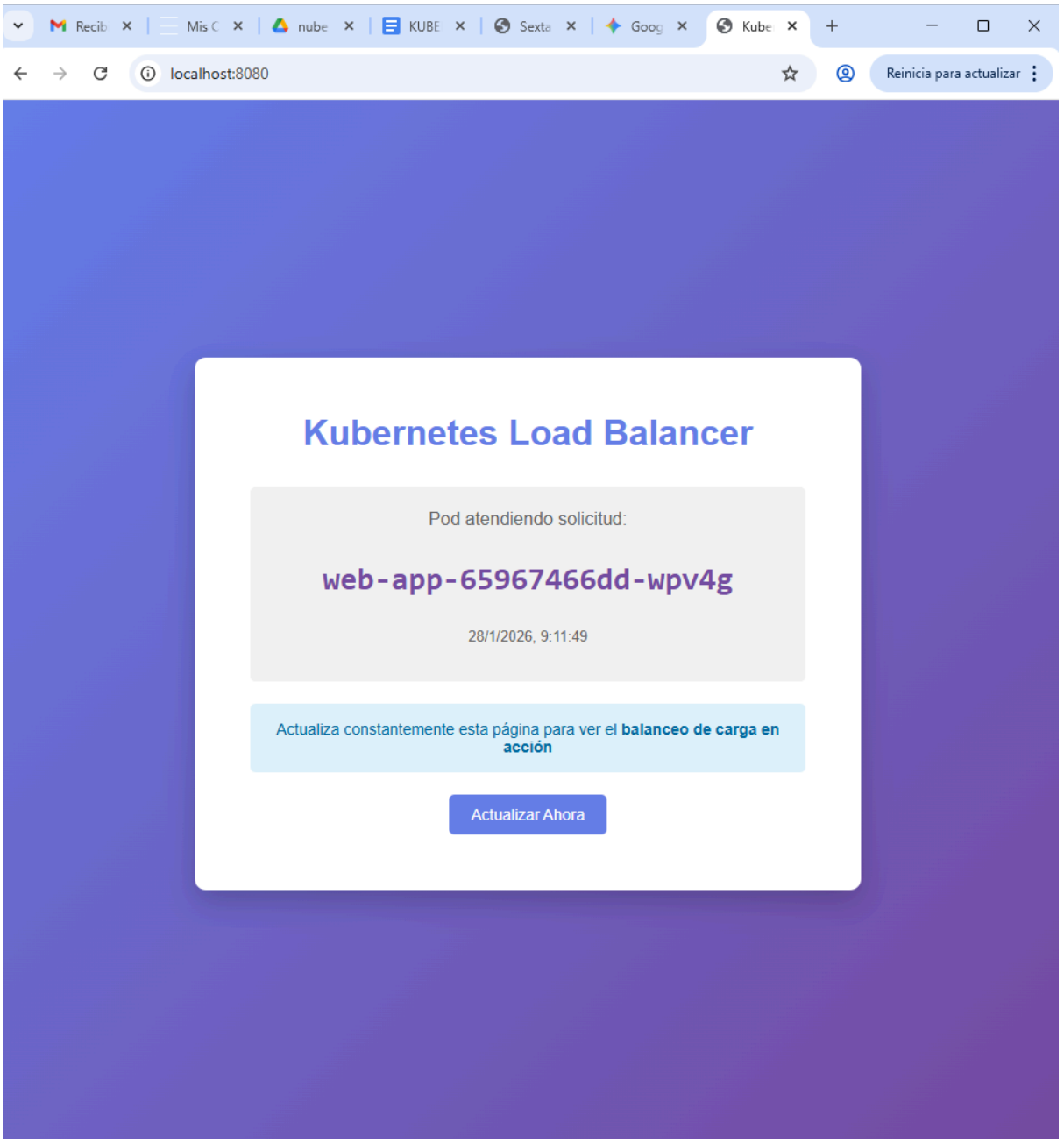
Bash

```
for i in {1..10}; do
  echo "Petición $i:"
  curl -s http://localhost:8080/pod-info | python3 -m json.tool | grep pod_name
  sleep 1
done
```

```
pepe@A6Alumno06:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ for i in {1..10}; do
  echo "Petición $i:"
  curl -s http://localhost:8080/pod-info | python3 -m json.tool | grep pod_name
  sleep 1
done
Petición 1:
    "pod_name": "web-app-65967466dd-wpv4g",
Petición 2:
    "pod_name": "web-app-65967466dd-wpv4g",
Petición 3:
    "pod_name": "web-app-65967466dd-wpv4g",
Petición 4:
    "pod_name": "web-app-65967466dd-wpv4g",
Petición 5:
    "pod_name": "web-app-65967466dd-wpv4g",
Petición 6:
    "pod_name": "web-app-65967466dd-wpv4g",
Petición 7:
    "pod_name": "web-app-65967466dd-wpv4g",
Petición 8:
    "pod_name": "web-app-65967466dd-wpv4g",
Petición 9:
    "pod_name": "web-app-65967466dd-wpv4g",
Petición 10:
    "pod_name": "web-app-65967466dd-wpv4g",
pepe@A6Alumno06:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$
```

2. **Navegador:** Abre `http://localhost:8080`. Refresca la página varias veces. Verás que el nombre del Pod cambia .


localhost:8080
Reinicia para actualizar
Kubernetes Load Balancer
Pod atendiendo solicitud:
web-app-65967466dd-wpv4g
28/1/2026, 9:11:34
Actualiza constantemente esta página para ver el balanceo de carga en acción
Actualizar Ahora

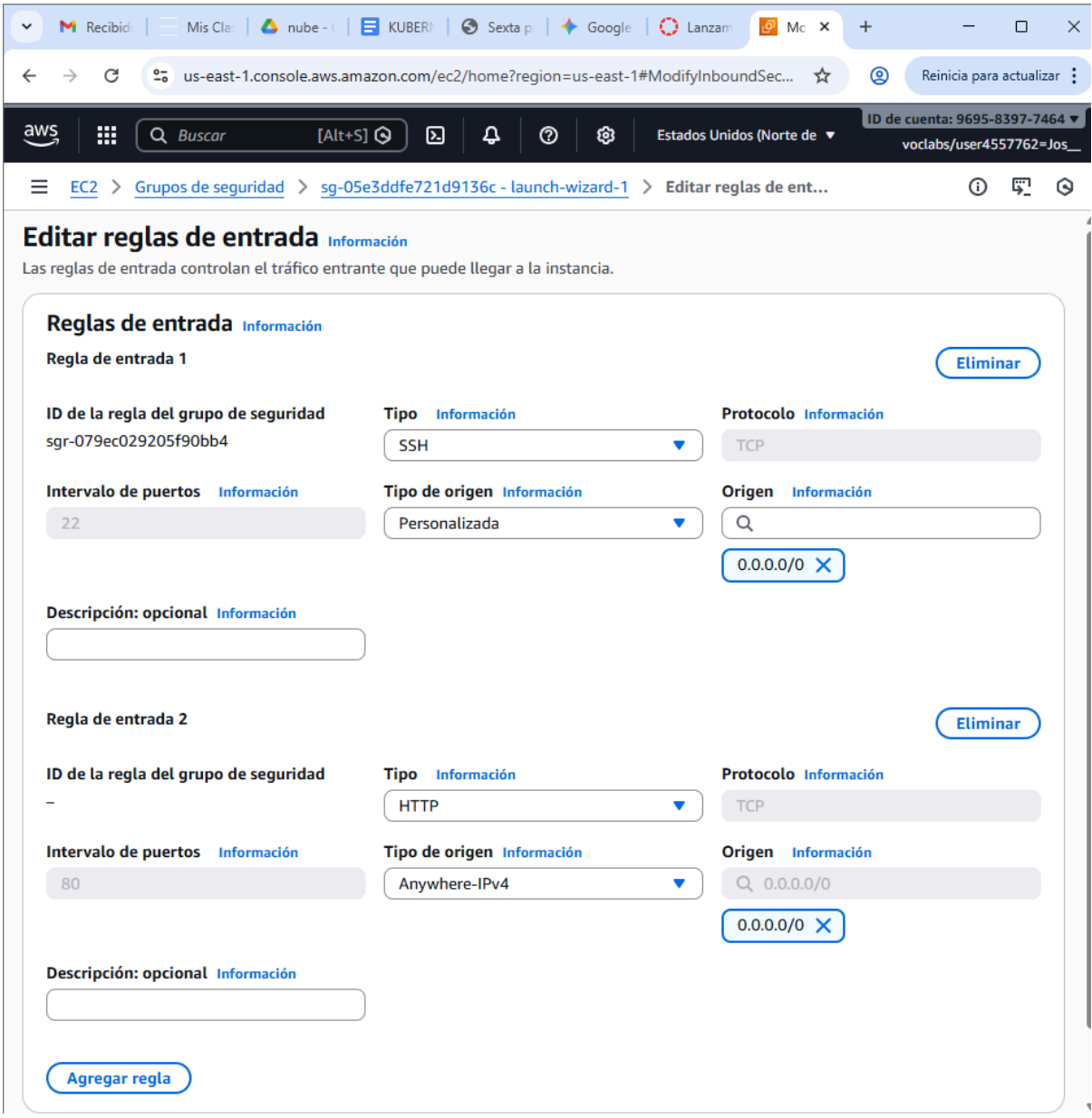


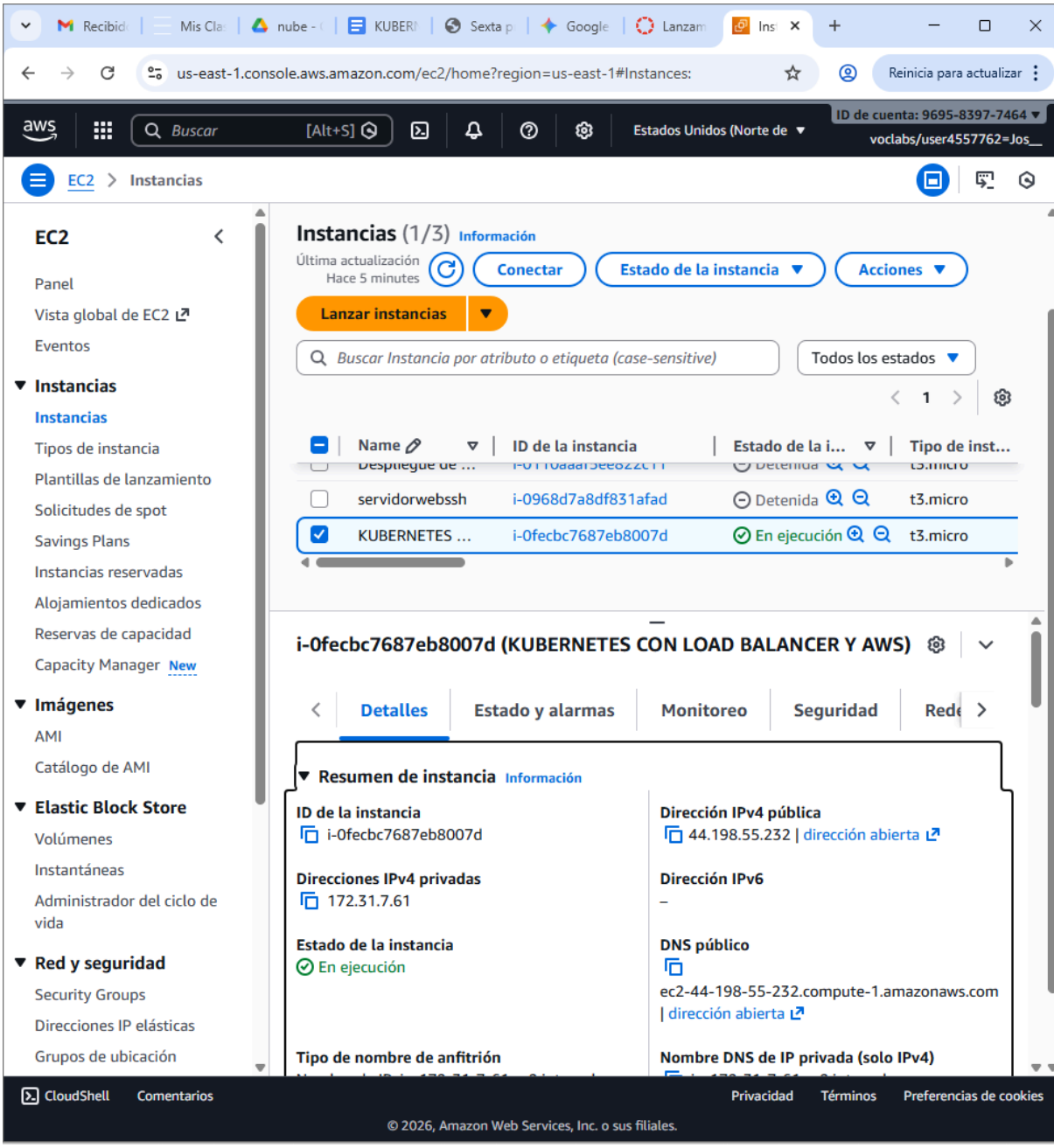
PARTE 4: Configuración en AWS

Ahora vamos a simular que un servidor externo (AWS EC2) se conecta a nuestro clúster local.

1. Crear Security Group (Consola AWS):
    - Ve a **EC2 > Security Groups > Create security group**.
    - Nombre: `kubernetes-aws-sg`.
    - Inbound Rules (Reglas de entrada):
        - SSH (TCP 22) -> Source: `0.0.0.0/0` (o tu IP).
        - HTTP (TCP 80) -> Source: `0.0.0.0/0`.
2. Lanzar Instancia EC2:

- **AMI:** Ubuntu 24.04 LTS.
- **Instance Type:** `t3.micro` (Elegible para Free Tier).
- **Key Pair:** Selecciona tu clave `.pem`.
- **Security Group:** Selecciona `kubernetes-aws-sg`.
- Lanza la instancia.





Preparar la instancia EC2: Conéctate desde tu terminal WSL (necesitas la IP pública de la instancia):
Bash
ssh -i ~/.ssh/tu-clave-aws.pem ubuntu@TU_IP_PUBLICA_EC2

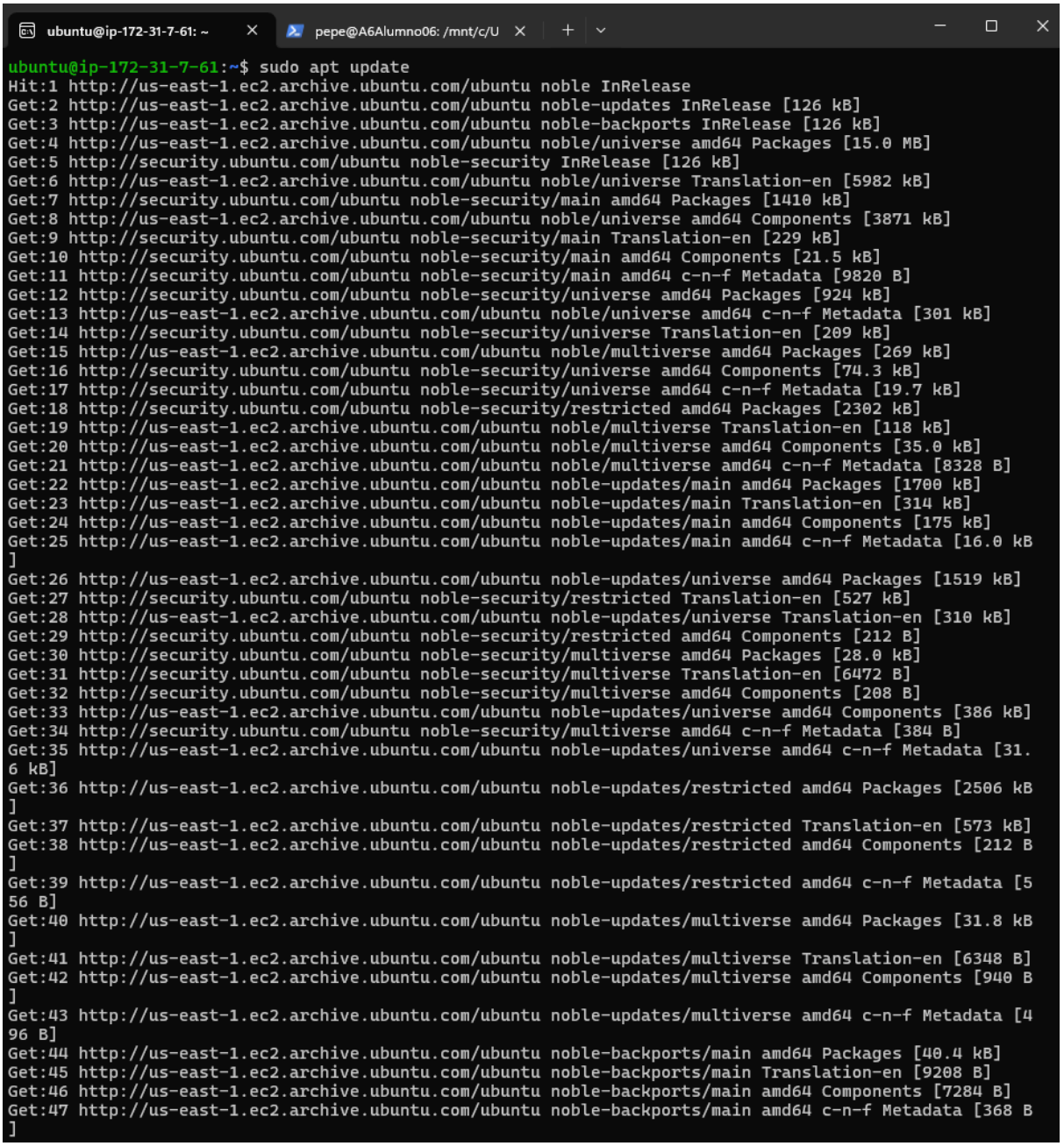
```
pepe@A6Alumno06:~$ ssh -i ~/.ssh/labuser_pepe.pem ubuntu@44.198.55.232
The authenticity of host '44.198.55.232 (44.198.55.232)' can't be established.
ED25519 key fingerprint is SHA256:1DnRRDDfxdRPjmv7hzuxClFMIrG8kC/ax3BuxND0RcU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '44.198.55.232' (ED25519) to the list of known host
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1018-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Jan 28 09:23:42 UTC 2026

  System load:  0.0               Temperature:           -273.1 C
  Usage of /:   26.0% of 6.71GB   Processes:             108
  Memory usage: 23%               Users logged in:       0
  Swap usage:   0%                IPv4 address for ens5: 172.31.7.61

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
```

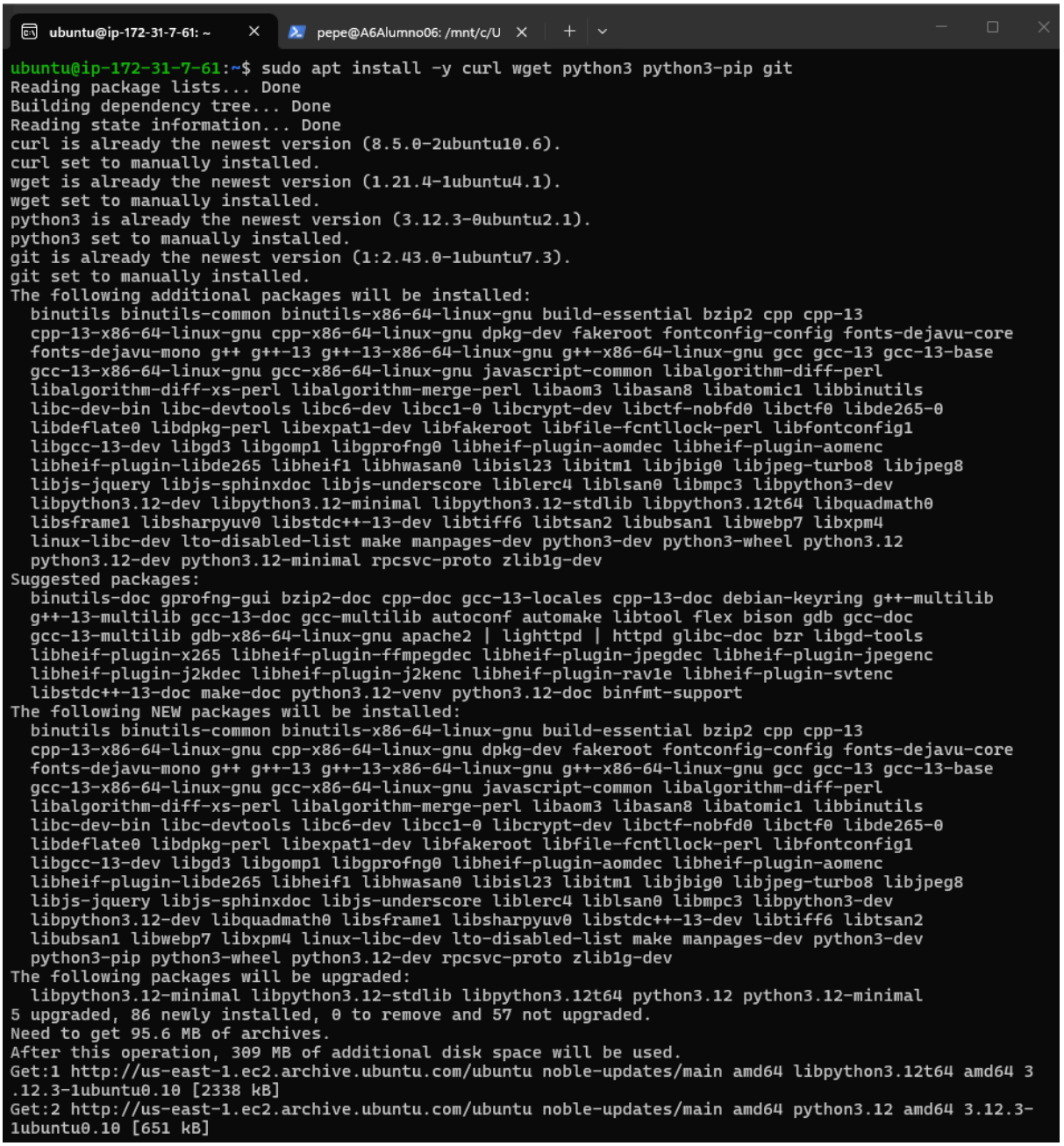
Una vez dentro de EC2, instala lo necesario:

Bash

sudo apt update

sudo apt install -y curl wget python3 python3-pip git

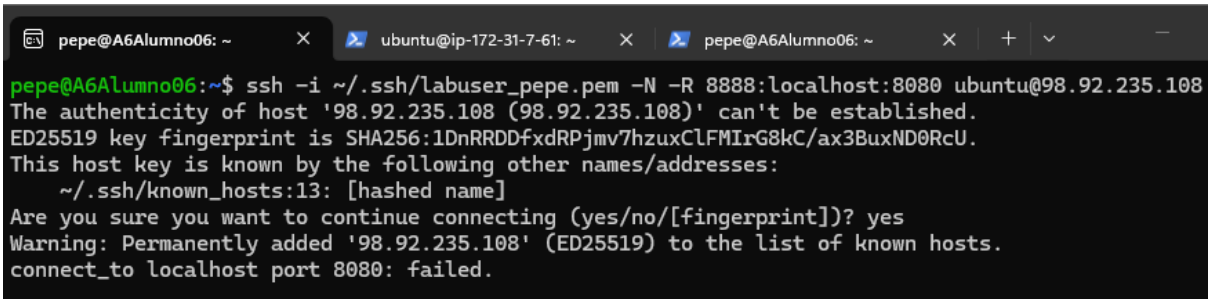

ubuntu@ip-172-31-7-61: ~
pepe@A6Alumno06: /mnt/c/U
ubuntu@ip-172-31-7-61:~$ sudo apt update
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:5 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1410 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [229 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [9820 B]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [924 kB]
Get:13 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [209 kB]
Get:15 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.3 kB]
Get:17 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [19.7 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2302 kB]
Get:19 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:20 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:21 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:22 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1700 kB]
Get:23 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [314 kB]
Get:24 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:25 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.0 kB]
Get:26 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1519 kB]
Get:27 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [527 kB]
Get:28 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [310 kB]
Get:29 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:30 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [28.0 kB]
Get:31 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [6472 B]
Get:32 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Get:33 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:34 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [384 B]
Get:35 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [31.6 kB]
Get:36 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2506 kB]
Get:37 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [573 kB]
Get:38 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:39 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 c-n-f Metadata [556 B]
Get:40 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [31.8 kB]
Get:41 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [6348 B]
Get:42 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:43 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 c-n-f Metadata [496 B]
Get:44 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports/main amd64 Packages [40.4 kB]
Get:45 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports/main Translation-en [9208 B]
Get:46 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7284 B]
Get:47 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports/main amd64 c-n-f Metadata [368 B]



PARTE 5: Conectar Kubernetes Local con AWS (Túnel SSH)

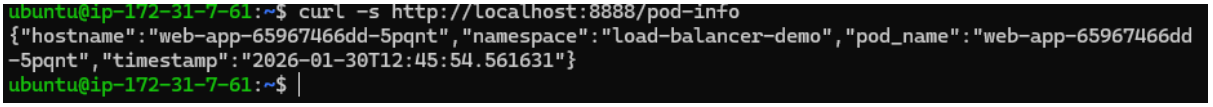
Este es el paso mágico. Vamos a hacer que el puerto 8888 de la máquina AWS responda con tu Kubernetes local.

Crear el Túnel (Desde tu WSL Local): Abre una **nueva terminal** en WSL. Ejecuta esto (reemplaza con tus datos):
Bash
ssh -i ~/.ssh/tu-clave-aws.pem -N -R 8888:localhost:8080 ubuntu@TU_IP_PUBLICA_EC2



1.
    - `-R 8888:localhost:8080`: Lo que llegue al puerto 8888 de AWS, envíalo a mi `localhost:8080` (donde está el port-forward de Kubernetes).
    - **Nota:** Debes mantener abierto tanto el `kubectl port-forward` como este túnel SSH.

Verificar desde AWS (Terminal EC2): En la terminal donde estás conectado a EC2, prueba:
Bash
curl -s http://localhost:8888/pod-info



2. ¡Deberías recibir el JSON de tu pod local!

Script de prueba de carga en AWS: En la instancia EC2, crea y ejecuta este script para visualizar el balanceo:
Bash

```
cat > ~/test-kubernetes-lb.sh << 'EOF'
#!/bin/bash
echo "=== Prueba Kubernetes Load Balancer desde AWS ==="
echo ""
declare -A pods_count
for i in {1..15}; do
  response=$(curl -s http://localhost:8888/pod-info)
  pod_name=$(echo $response | python3 -c "import sys, json;
print(json.load(sys.stdin)['pod_name'])" 2>/dev/null)

  if [ -z "$pod_name" ]; then pod_name="ERROR"; fi

  echo "Petición $i -> Pod: $pod_name"
  pods_count[$pod_name]=$((${pods_count[$pod_name]:-0} + 1))
  sleep 1
done
echo ""
echo "=== Resumen Balanceo ==="
for pod in "${!pods_count[@]}"; do
  echo "Pod $pod: ${pods_count[$pod]} peticiones"
```

done
EOF

chmod +x ~/test-kubernetes-lb.sh
~/test-kubernetes-lb.sh

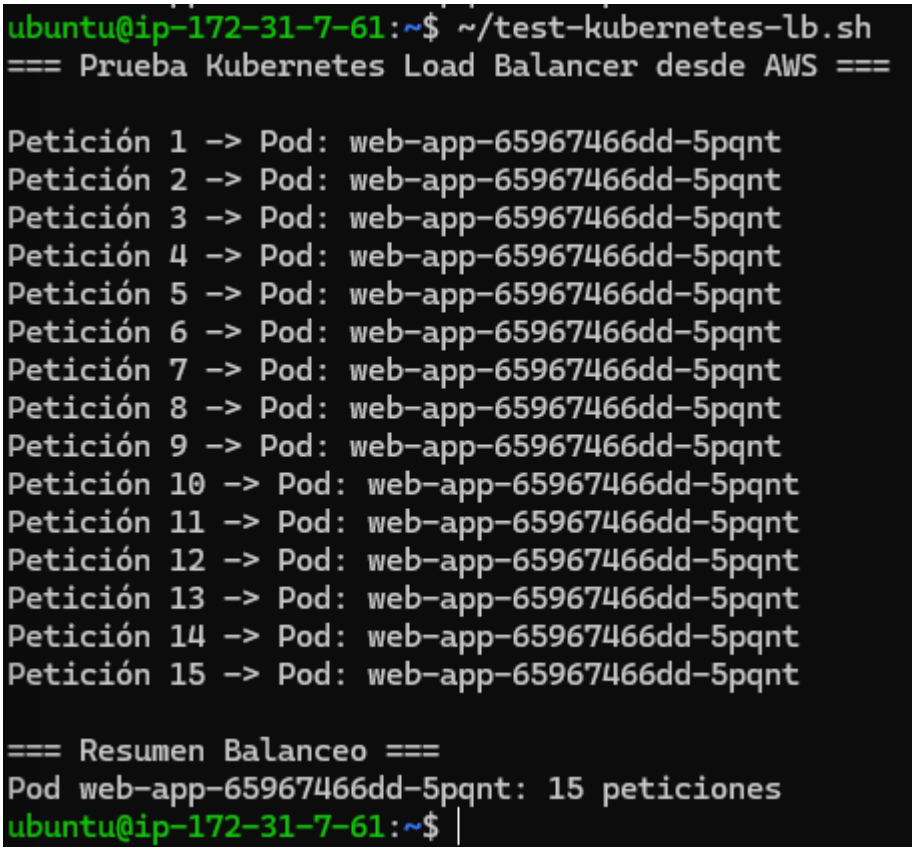
```
ubuntu@ip-172-31-7-61:~$ cat > ~/test-kubernetes-lb.sh << 'EOF'
#!/bin/bash
echo "=== Prueba Kubernetes Load Balancer desde AWS ==="
echo ""
declare -A pods_count
for i in {1..15}; do
  response=$(curl -s http://localhost:8888/pod-info)
  pod_name=$(echo $response | python3 -c "import sys, json; print(json.load(sys.stdin)['pod_name'])" 2
>/dev/null)

  if [ -z "$pod_name" ]; then pod_name="ERROR"; fi

  echo "Petición $i -> Pod: $pod_name"
  pods_count[$pod_name]=$((${pods_count[$pod_name]:-0} + 1))
  sleep 1
done
echo ""
echo "=== Resumen Balanceo ==="
for pod in "${!pods_count[@]}"; do
  echo "Pod $pod: ${pods_count[$pod]} peticiones"
done
EOF

chmod +x ~/test-kubernetes-lb.sh
~/test-kubernetes-lb.sh
=== Prueba Kubernetes Load Balancer desde AWS ===

Petición 1 -> Pod: web-app-65967466dd-5pqnt
Petición 2 -> Pod: web-app-65967466dd-5pqnt
Petición 3 -> Pod: web-app-65967466dd-5pqnt
Petición 4 -> Pod: web-app-65967466dd-5pqnt
Petición 5 -> Pod: web-app-65967466dd-5pqnt
Petición 6 -> Pod: web-app-65967466dd-5pqnt
Petición 7 -> Pod: web-app-65967466dd-5pqnt
Petición 8 -> Pod: web-app-65967466dd-5pqnt
Petición 9 -> Pod: web-app-65967466dd-5pqnt
Petición 10 -> Pod: web-app-65967466dd-5pqnt
Petición 11 -> Pod: web-app-65967466dd-5pqnt
Petición 12 -> Pod: web-app-65967466dd-5pqnt
Petición 13 -> Pod: web-app-65967466dd-5pqnt
Petición 14 -> Pod: web-app-65967466dd-5pqnt
Petición 15 -> Pod: web-app-65967466dd-5pqnt

=== Resumen Balanceo ===
Pod web-app-65967466dd-5pqnt: 15 peticiones
ubuntu@ip-172-31-7-61:~$
```


PARTE 6: Escalado Dinámico

Vamos a ver cómo Kubernetes reacciona en tiempo real.

Escalar (Desde WSL Local): Aumenta los pods de 3 a 5.
Bash
kubectl scale deployment web-app -n load-balancer-demo --replicas=5

1. Verifica que se creen: `kubectl get pods -n load-balancer-demo`.
2. **Re-testear (Desde AWS EC2):** Vuelve a ejecutar `~/test-kubernetes-lb.sh` en la instancia EC2. Verás que ahora el tráfico se reparte entre 5 pods diferentes, no solo 3.



PARTE 7: Monitoreo y Observabilidad

Comandos útiles para inspeccionar tu clúster desde WSL:

- **Ver uso de recursos:** `kubectl top pods -n load-balancer-demo` (CPU/RAM).
- **Ver logs de un pod:** `kubectl logs -n load-balancer-demo -l app=web-app -f`.
- **Ver detalles técnicos:** `kubectl describe deployment web-app -n load-balancer-demo`.

PARTE 8: Eliminar Recursos (Limpieza)

Muy importante para evitar costes en AWS y mantener tu PC limpio.

Limpiar Kubernetes (WSL):
Bash
kubectl delete namespace load-balancer-demo

1.
2. **Eliminar Instancia EC2 (AWS Console):** Ve a la consola, selecciona tu instancia -> **Instance State -> Terminate**.

Detener k3s (WSL):
Bash
sudo systemctl stop k3s
# O para desinstalar completamente:
sudo /usr/local/bin/k3s-uninstall.sh